P0019r3 : Atomic View

Project:	ISO JTC1/SC22/WG21: Programming Language C++
Number:	P0019r3
Date:	2016-10-14
Reply-to:	hcedwar@sandia.gov
Author:	H. Carter Edwards
Contact:	hcedwar@sandia.gov
Author:	Hans Boehm
Contact:	hboehm@google.com
Author:	Olivier Giroux
Contact:	ogiroux@nvidia.com
Author:	James Reus
Contact:	reus1@llnl.gov
Audience:	Library Evolution, SG1 Concurrency
URL:	https://github.com/kokkos/ISO-CPP-Papers/blob/master/P0019.rst

Revision History

P0019r3

- Align proposal with content of corresponding sections in N5131, 2016-07-15.
- Remove the *one root wrapping constructor* requirement from **atomic_array_view**.
- Other minor revisions responding to feedback from SG1 @ Oulu.

Overview / Motivation / Discussion

This paper proposes an extension to the atomic operations library [atomics] for atomic operations applied to non-atomic objects.

Naming

Feedback from Library Evolution Working Group (LEWG) on P0009r0, Polymorphic Multidimensional Array View, noted that the term *view* has the connotation of read-only. In response the P0009r0 *array_view* name has been revised to **array_ref** in P0009r1. The proposed names **atomic_view** and **atomic_array_view** may have the same feedback from LEWG, potentially resulting in a similar naming revision.

The current **29.2 Header <atomic> synopsis** contains the following.

```
namespace std {
  template< class T > struct atomic;
  template<> struct atomic< integral >;
  template< class T > struct atomic<T*>;
}
```

The current proposal introduces the following additional types.

```
namespace std {
namespace experimental {

  template< class T > struct atomic_view;
  template<> struct atomic_view< integral >;
  template< class T > struct atomic_view<T*>;
  template< class T > struct atomic_array_view;

}}
```

An alternative naming convention is to follow the **atomic<T*>** partial specialization strategy for naming.

```
namespace std {
namespace experimental {

    template< class T > struct atomic< T & > ;
    template<> struct atomic< integral & >;
    template< class T > struct atomic< T * & >;
    template< class T > struct atomic< T [] >;

}}
```

Atomic Operations on a Single Non-atomic Object

An *atomic view* is used to perform atomic operations on referenced non-atomic object. The intent is for *atomic view* to provide the best-performing implementation of atomic operations for the non-atomic object type. All atomic operations performed through an *atomic view* on a referenced non-atomic object are atomic with respect to any other *atomic view* that references the same object, as defined by equality of pointers to that object. The intent is for atomic operations to directly update the referenced object. The *atomic view wrapping constructor* may acquire a resource, such as a lock from a collection of address-sharded locks, to perform atomic operations. Such *atomic view* objects are not lock-free and not address-free. When such a resource is necessary subsequent copy and move constructors and assignment operators may reduce overhead by copying or moving the previously acquired resource as opposed to re-acquiring that resource.

Introducing concurrency within legacy codes may require replacing operations on existing non-atomic objects with atomic operations such that the non-atomic object cannot be replaced with a **atomic** object.

An object may be heavily used non-atomically in well-defined phases of an application. Forcing such objects to be exclusively **atomic** would incur an unnecessary performance penalty.

Atomic Operations on Members of a Very Large Array

High performance computing (HPC) applications use very large arrays. Computations with these arrays typically have distinct phases that allocate and initialize members of the array, update members of the array, and read members of the array. Parallel algorithms for initialization (e.g., zero fill) have non-conflicting access when assigning member values. Parallel algorithms for updates have conflicting access to members which must be guarded by atomic operations. Parallel algorithms with read-only access require best-performing streaming read access, random read access, vectorization, or other guaranteed non-conflicting HPC pattern.

An *atomic array view* is used to perform atomic operations on the non-atomic members of the referenced array. The intent is for *atomic array view* to provide the best-performing implementation of atomic operations for the members of the array.

Wrapping Constructor Error Response

The *wrapping constructor* of an atomic view is responsible for detecting potential errors associated with wrapping a non-atomic object. For example, if the object does satisfy alignment requirements or resides in memory where atomic operations are not supported (e.g, GPU registers). The wrapping constructor's response to such errors is to throw an exception, an alternative response is to abort.

Proposal

add to 29.2 Header <atomic> synopsis

```
namespace std {
namespace experimental {

    template< class T > struct atomic_view ;
    template<> struct atomic_view< integral >;
    template< class T > struct atomic_view< T * >;
    template< class T > struct atomic_array_view ;

}}
```

add to 29.5 Atomic Types

```
template< class T > struct atomic_view {
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( T , memory_order = memory_order_seq_cst ) const noexcept;
    T load( memory_order = memory_order_seq_cst ) const noexcept;
    operator T() const noexcept ;
    T exchange( T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( T& , T , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( T& , T , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( T&, T, memory_order = memory_order_seq_cst ) const noexcept;

    ~atomic_view();
    constexpr atomic_view() noexcept ;
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( atomic_view && ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    T operator=(T) const noexcept ;

    explicit atomic_view( T & obj ); // wrapping constructor
    explicit constexpr operator bool () const noexcept; // wraps
};

template<> struct atomic_view< integral > {
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( integral , memory_order = memory_order_seq_cst ) const noexcept;
    integral load( memory_order = memory_order_seq_cst ) const noexcept;
    operator integral () const noexcept ;
    integral exchange( integral , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( integral & , integral , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( integral & , integral , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( integral & , integral , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( integral &, integral , memory_order = memory_order_seq_cst ) const noexcept;

    integral fetch_add( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_sub( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_and( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_or( integral , memory_order = memory_order_seq_cst) const noexcept;
    integral fetch_xor( integral , memory_order = memory_order_seq_cst) const noexcept;

    ~atomic_view();
    constexpr atomic_view() noexcept ;
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( atomic_view && ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    integral operator=( integral ) const noexcept ;

    explicit atomic_view( integral & obj ); // wrapping constructor
    explicit constexpr operator bool () const noexcept; // wraps

    integral operator++(int) const noexcept;
    integral operator--(int) const noexcept;
    integral operator++() const noexcept;
    integral operator--() const noexcept;
```

```
    integral operator+=( integral ) const noexcept;
    integral operator-=( integral ) const noexcept;
    integral operator&=( integral ) const noexcept;
    integral operator|=( integral ) const noexcept;
    integral operator^=( integral ) const noexcept;
};

template<class T> struct atomic_view< T * > {
    static constexpr size_t required_alignment = implementation-defined ;
    static constexpr bool is_always_lock_free = implementation-defined ;
    bool is_lock_free() const noexcept;
    void store( T * , memory_order = memory_order_seq_cst ) const noexcept;
    T * load( memory_order = memory_order_seq_cst ) const noexcept;
    operator T * () const noexcept ;
    T * exchange( T * , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_weak( T * & , T * , memory_order , memory_order ) const noexcept;
    bool compare_exchange_strong( T * & , T * , memory_order , memory_order ) const noexcept;
    bool compare_exchange_weak( T * & , T * , memory_order = memory_order_seq_cst ) const noexcept;
    bool compare_exchange_strong( T * &, T * , memory_order = memory_order_seq_cst ) const noexcept;

    T * fetch_add( ptrdiff_t , memory_order = memory_order_seq_cst) const noexcept;
    T * fetch_sub( ptrdiff_t , memory_order = memory_order_seq_cst) const noexcept;

    ~atomic_view();
    constexpr atomic_view() noexcept ;
    atomic_view( atomic_view && ) noexcept ;
    atomic_view( const atomic_view & ) noexcept ;
    atomic_view & operator = ( atomic_view && ) noexcept ;
    atomic_view & operator = ( const atomic_view & ) noexcept ;
    T * operator=( T * ) const noexcept ;

    explicit atomic_view( T * & obj ); // wrapping constructor
    explicit constexpr operator bool () const noexcept; // wraps

    T * operator++(int) const noexcept;
    T * operator--(int) const noexcept;
    T * operator++() const noexcept;
    T * operator--() const noexcept;
    T * operator+=( ptrdiff_t ) const noexcept;
    T * operator-=( ptrdiff_t ) const noexcept;
};

template< class T > struct atomic_array_view {

    static constexpr size_t required_alignment = implementation defined ;
    static constexpr bool is_always_lock_free = implementation defined ;
    bool is_lock_free() const noexcept ;

    explicit constexpr operator bool() const noexcept ;

    atomic_array_view( T * , size_t ); // wrapping constructor

    constexpr atomic_array_view() noexcept ;
    atomic_array_view( atomic_array_view && ) noexcept ;
    atomic_array_view( const atomic_array_view & ) noexcept ;
    atomic_array_view & operator = ( atomic_array_view && ) noexcept ;
    atomic_array_view & operator = ( const atomic_array_view & ) noexcept ;
    ~atomic_array_view();

    size_t size() const noexcept ;

    atomic_view<T> operator[]( size_t ) const noexcept;
```

};

1 There are generic class templates atomic<T>, atomic_view<T>, and atomic_array_view<T>.

add 29.6.6 Requirements for operations on atomic view types

In the following operation definitions:

- an *A* refers to one of the atomic view types.
- a *C* refers to its corresponding non-atomic type
- an *M* refers to type of other argument for arithmetic operations. For integral atomic view types, *M* is *C*. For atomic view address types, *M* is **std::ptrdiff_t**.

static constexpr bool A::is_always_lock_free = *implementation-defined* **;**

Is true if the atomic operations are always lock-free, and false otherwise.

bool A::is_lock_free() const noexcept;

Returns: **true** if the atomic operations are lock-free, **false** otherwise.

static constexpr size_t required_alignment = *implementation-defined* **;**

The required alignment of an object to be referenced by an atomic view, which is at least `align_of(C)`. [Note: An architecture may support lock-free atomic operations on objects of type *C* only if those objects meet a required alignment. The intent is for *atomic_view* to provide lock-free atomic operations whenever possible. For example, an architecture may be able to support lock-free operations on **std::complex<double>** only if aligned to 16 bytes and not 8 bytes. - end note]

constexpr A::A() noexcept;

Effects: ***this** does not reference an object.

A::A(C & object);

This *wrapping constructor* constructs an *atomic view* that references the non-atomic *object*. Atomic operations applied to *object* through a referencing *atomic view* are atomic with respect to atomic operations applied through any other *atomic view* that references that *object*.

Requires: The referenced non-atomic *object* shall be aligned to **required_alignment**. The lifetime (3.8) of ***this** shall not exceed the lifetime of the referenced non-atomic object. While any **atomic_view** instance exists that references *object* all accesses of that *object* shall exclusively occur through those **atomic_view** instances. If the referenced *object* is of a class or aggregate type then members of that object shall not be concurrently wrapped by an **atomic_view** object. The referenced *object* shall not be a member of an array that is wrapped by an **atomic_array_view** .

Effects: ***this** references the non-atomic *object*. [Note: The *wrapping constructor* may acquire a shared resource, such as a lock associated with the referenced object, to enable atomic operations applied to the referenced non-atomic object. - end note]

Throws (aborts): If member atomic operation functions cannot be applied to the referenced *object* then the *wrapping* constructor shall throw (abort). [Note: For example, if the referenced object is not properly aligned or has automatic storage duration within an accelerator coprocessor (*e.g.*, a GPGPU) execution context. - end note] If the *wrapping constructor* attempts and fails to acquire resources such as a lock associated with the referenced *object* then the *wrapping constructor* shall throw (abort).

A::A(A && rhs) noexcept ;
A & A::operator = (A && rhs) noexcept ;

Effects: If *rhs* references an object then ***this** references that object **rhs** no longer references an object, otherwise ***this** does not reference an object. If *rhs* also references an acquired shared resource then ***this** references that shared resource and **rhs** no longer references that shared resource, otherwise ***this** does not reference a shared resource.

A::A(A const & rhs) noexcept ;
A & A::operator = (A const & rhs) noexcept ;

Effects: If *rhs* references an object then ***this** references the same object, otherwise ***this** does not reference an object.

If *rhs* also references a shared resource then ***this** references that shared resource, otherwise ***this** does not reference a shared resource.

A::~A() noexcept ;

Effects: If ***this** references an acquired shared resource then ***this** releases that shared resource.

explicit constexpr A::operator bool () const noexept ;

Returns: **true** if ***this** references a non-atomic object, otherwise **false**.

void A::atomic_store(C::desired, memory_order order = memory_order_seq_cst) const noexcept;

Requires: ***this** references an object. The order argument shall not be memory_order_consume, memory_order_acquire, nor memory_order_acq_rel.

Effects: Atomically replaces the value referenced by ***this** with the value of *desired*. Memory is affected according to the value of order.

C A::operator=(C desired) const noexcept;

Effects: As if by **A::store(desired)**.

Returns: *desired*.

void A::atomic_load(memory_order order = memory_order_seq_cst) const noexcept;

Requires: ***this** references an object. The order argument shall not be memory_order_release nor memory_order_acq_rel.

Effects: Memory is affected according to the value of order.

Returns: Atomically returns the value referenced by ***this** .

A::operator C() const noexcept;

Effects: As if by **A::load()**.

C A::exchange(C desired, memory_order order = memory_order_seq_cst) noexcept;

Requires: ***this** references an object.

Effects: Atomically replaces the value referenced by ***this** with *desired*. Memory is affected according to the value of *order*. These operations are atomic read-modify-write operations (1.10).

Returns: Atomically returns the value referenced by ***this** immediately before the effects.

bool A::compare_exchange_weak(C & expected, C desired, memory_order success, memory_order failure) const noexcept;
bool A::compare_exchange_strong(C & expected, C desired, memory_order success, memory_order failure) const noexcept;
bool A::compare_exchange_weak(C & expected, C desired,memory_order order = memory_order_seq_cst) const noexcept;
bool A::compare_exchange_strong(C & expected, C desired, memory_order order = memory_order_seq_cst) const noexcept;

Requires: ***this** references an object. The *failure* argument shall not be memory_order_release nor memory_order_acq_rel. The *failure* argument shall be no stronger than the *success* argument.

Effects: Retrieves the value in *expected*. It then atomically compares the contents of the memory referenced by ***this** for equality with that previously retrieved from *expected*, and if true, replaces the contents of the memory referenced by ***this** with that in *desired*. If and only if the comparison is true, memory is affected according to the value of success, and if the comparison is false, memory is affected according to the value of failure. When only one memory_order argument is supplied, the value of success is *order*, and the value of failure is *order* except that a value of memory_order_acq_rel shall be replaced by the value memory_order_acquire and a value of memory_order_release shall be replaced by the value memory_order_relaxed. If and only if the comparison is false then, after the atomic operation, the contents of the memory in *expected* are replaced by the value read from memory referenced by ***this** during the atomic comparison. If the operation returns true, these operations are atomic read-modify-write operations (1.10) on the memory referenced by ***this**. Otherwise, these operations are atomic load operations on that memory.

Returns: The result of the comparison.

[Note: See 29.6.5 p24-27 notes and remarks. --end node]

A::fetch_*key*(M operand, memory_order order = memory_order_seq_cst) const noexcept;

Requires: ***this** references an object.

Effects: Atomically replaces the value referenced by ***this** with the result of the computation applied to the value referenced by ***this** and the given operand. Memory is affected according to the value of *order*. These operations are atomic read-modify-write operations (1.10).

Returns: Atomically, the value referenced by ***this** immediately before the effects.

Remark: For signed integer types, arithmetic is defined to use two's complement representation. There are no undefined results. For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.

A::operator *op* **=(M operand) const noexcept;**

Effects: As if by fetch_key (operand).

Returns: fetch_key (operand) op operand.

A::operator++(int) const noexcept;

Returns: fetch_add(1).

A::operator--(int) const noexcept;

Returns: fetch_sub(1).

A::operator++() const noexcept;

Effects: As if by fetch_add(1).

Returns: fetch_add(1) + 1.

C::operator--() const noexcept;

Effects: As if by fetch_sub(1).

Returns: fetch_sub(1) - 1.

add 29.6.7 Requirements for operations on atomic array view types

In the following operation definitions:

- an *A* refers to one of the atomic array view types.
- a *C* refers to its corresponding non-atomic type

static constexpr bool A::is_always_lock_free = *implementation-defined* **;**

Is true if the atomic operations are always lock-free, and false otherwise.

bool A::is_lock_free() const noexcept;

Returns: **true** if atomic operations are lock-free, **false** otherwise.

static constexpr size_t required_alignment = *implementation-defined* **;**

The required alignment of an array to be referenced by an atomic view, which is at least `align_of(C)`.

Remark: An architecture may support lock-free atomic operations on objects of type *C* only if those objects meet a required alignment. The intent is for *atomic_array_view* to provide lock-free atomic operations whenever possible. [Note: For example, an architecture may be able to support lock-free operations on **std::complex<double>** only if aligned to 16 bytes and not 8 bytes. - end note]

constexpr A::A() noexcept;

Effects: ***this** does not reference an array and therefore **operator bool()** == **false**.

A::A(C * array , size_t length);

This *wrapping constructor* constructs an *atomic_array_view* that references an array of non-atomic elements spanning `[array..array+length)`.

Requires: The referenced non-atomic array shall be aligned to **required_alignment**. The lifetime (3.8) of ***this** shall not exceed the lifetime of the referenced non-atomic array. All **atomic_array_view** instances that reference any element of the array shall reference the same span of the array. As long as any **atomic_array_view** instance exists that references array all accesses to members of that array shall exclusively occur through those **atomic_array_view** instances. No element of array is concurrently *wrap constructed* by an **atomic_view**.

Effects: ***this** references the non-atomic array. Atomic operations on members of array are atomic with respect to atomic operations on members referenced through any other **atomic_array_view** instance. [Note: The *wrapping constructor* may acquire shared resources, such as a locks associated with the referenced array, to enable atomic operations applied to the referenced non-atomic members of referenced array. - end note]

Throws (aborts): If member atomic operation functions cannot be applied to the referenced mmebers of *array* then the *wrapping* constructor shall throw (abort). [Note: For example, if the referenced array is not properly aligned or has automatic storage duration within an accelerator coprocessor (*e.g.*, a GPGPU) execution context. - end note] If the *wrapping constructor* attempts and fails to acquire resources such as a lock associated with the referenced *object* then the *wrapping constructor* shall throw (abort).

A::A(A && rhs) noexcept ;
A & A::operator = (A && rhs) noexcept ;

Effects: If *rhs* references an array then ***this** references that array and **rhs** no longer references an array, otherwise ***this** does not reference an array. If *rhs* also references acquired shared resources then ***this** references those shared resources and **rhs** no longer references those shared resources, otherwise ***this** does not reference shared resources.

A::A(A const & rhs) noexcept ;
A & A::operator = (A const & rhs) noexcept ;

Effects: If *rhs* references an array then ***this** references the same array, otherwise ***this** does not reference an array. If *rhs* also references shared resources then ***this** references those shared resources, otherwise ***this** does not reference shared resources.

A::~A() noexcept ;

Effects: If ***this** references a acquired shared resources then ***this** releases those shared resources.

explicit constexpr A::operator bool () const noexept ;

Returns: **true** if ***this** references a non-atomic array, otherwise **false**.

atomic_view<C> A::operator[](size_t i) const noexcept ;

Requires: **i** < **size()** and the lifetime of the returned **atomic_view** s shall not exceed the lifetime of the associated **atomic_array_view**. [Note: Analogous to the lifetime of an iterator with respect to the lifetime of the associated container. - end note]

Example usage:

```
// atomic array view wrapper constructor:
atomic_array_view<T> array( ptr , N );

// atomic operation on a member:
array[i].atomic-operation(...);

// atomic operations through a temporary value
// within a concurrent function:
atomic_array_view<T>::reference x = array[i];
x.atomic-operation-a(...);
x.atomic-operation-b(...);
```